

Unit4

Regularization: Bias Variance Tradeoff, L2 regularization, Early stopping, Dataset Augmentation, Parameter sharing and tying, Injecting noise at input, Ensemble methods, Dropout. Greedy Layerwise Pre-training, Better activation functions, Better weight initialization methods, Batch Normalization. Learning Vectorial Representations Of Words.

Regularization for Deep Learning

A central problem in machine learning.

- how to make an algorithm that will perform well not just on the training data, but also on new inputs.
- Many strategies used in machine learning are explicitly designed **to reduce the test error**, at the expense of increased training error.
- These strategies are known collectively as **regularization**.
- many forms of regularization available to the deep learning

Regularization for Deep Learning

- Regularization of an estimator works by trading increased bias for reduced variance.
- Many regularization approaches are based on limiting the capacity of models, such as neural networks, linear regression, or logistic regression, by adding a parameter norm penalty $\Omega(\boldsymbol{\vartheta})$ to the objective function J .

$$\tilde{J}(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) = J(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) + \alpha \Omega(\boldsymbol{\theta})$$

- Setting α to 0 results in no regularization.
- Larger values of α correspond to more regularization

L^2 Parameter Regularization (weight decay)

- L^2 parameter norm penalty commonly known as **weight decay**
- This regularization strategy drives the weights closer to the origin
- by adding regularization term $\Omega(\theta) = \frac{1}{2} \|\omega\|_2^2$ **to objective function.**
- also known as **ridge regression or Tikhonov regularization.**
- model has the following total objective function:

$$\tilde{J}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = \frac{\alpha}{2} \mathbf{w}^\top \mathbf{w} + J(\mathbf{w}; \mathbf{X}, \mathbf{y}),$$

with the corresponding parameter gradient

$$\nabla_{\mathbf{w}} \tilde{J}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = \alpha \mathbf{w} + \nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{X}, \mathbf{y}).$$

L^2 Parameter Regularization (weight decay)

- behavior of weight decay regularization by studying the gradient of the regularized objective function.
- To simplify presentation, we assume no bias parameter, so ϑ is just w .

$$w \leftarrow w - \epsilon (\alpha w + \nabla_w J(w; X, y)) .$$

$$w \leftarrow (1 - \epsilon\alpha)w - \epsilon \nabla_w J(w; X, y).$$

- addition of the weight decay term has modified the learning rule to multiplicatively shrink the weight vector by a constant factor on each step, just before performing the usual gradient update .
- This describes what happens in a single step.
- But what happens over the entire course of training?

L^2 Parameter Regularization (weight decay)

- analysis by making a quadratic approximation to the objective function in the neighborhood of the value of the weights **that obtains minimal unregularized training cost**,
- $w^* = \arg \min_w J(w)$.
- If the objective function is truly quadratic, as in the case of fitting a linear regression model with mean squared error, then the approximation is perfect.

$$\hat{J}(\boldsymbol{\theta}) = J(\boldsymbol{w}^*) + \frac{1}{2} (\boldsymbol{w} - \boldsymbol{w}^*)^\top \boldsymbol{H} (\boldsymbol{w} - \boldsymbol{w}^*),$$

- where H is the Hessian matrix of J with respect to w evaluated at w^* .

L^2 Parameter Regularization (weight decay)

$$\hat{J}(\boldsymbol{\theta}) = J(\boldsymbol{w}^*) + \frac{1}{2} (\boldsymbol{w} - \boldsymbol{w}^*)^\top \boldsymbol{H} (\boldsymbol{w} - \boldsymbol{w}^*),$$

- where H is the Hessian matrix of J with respect to w evaluated at w^* .
- There is no first-order term in this quadratic approximation, because w^* is defined to be a minimum, where the gradient vanishes.
- Likewise, because w^* is the location of a minimum of J ,
- we can conclude that H is positive semidefinite
- The minimum of $\hat{J}$ occurs where its gradient

$$\nabla_{\boldsymbol{w}} \hat{J}(\boldsymbol{w}) = \boldsymbol{H} (\boldsymbol{w} - \boldsymbol{w}^*)$$

is equal to 0.

L^2 Parameter Regularization (weight decay)

- by adding the weight decay gradient. We can now solve for the minimum of the regularized version of J^* .
- We use the variable $\tilde{\mathbf{w}}$ to represent the location of the minimum.

$$\alpha \tilde{\mathbf{w}} + \mathbf{H}(\tilde{\mathbf{w}} - \mathbf{w}^*) = 0$$

$$(\mathbf{H} + \alpha \mathbf{I}) \tilde{\mathbf{w}} = \mathbf{H} \mathbf{w}^*$$

$$\tilde{\mathbf{w}} = (\mathbf{H} + \alpha \mathbf{I})^{-1} \mathbf{H} \mathbf{w}^*.$$

- As α approaches 0, the regularized solution $\tilde{\mathbf{w}}$ approaches $\mathbf{w}^*$.
- But what happens as α grows?

$$Q\Lambda Q^T + \alpha I = Q(\Lambda + \alpha I)Q^T$$

Here, we first compute $\Lambda + \alpha I$:

- Since Λ is diagonal, αI adds α to each of the diagonal elements of Λ .
- Therefore, if $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$, then $\Lambda + \alpha I = \text{diag}(\lambda_1 + \alpha, \lambda_2 + \alpha, \dots, \lambda_n + \alpha)$.

$$Q(\Lambda + \alpha I)Q^T = Q\text{diag}(\lambda_1 + \alpha, \lambda_2 + \alpha, \dots, \lambda_n + \alpha)Q^T$$

$$Q\Lambda Q^T + \alpha I = Q(\Lambda + \alpha I)Q^T$$

L^2 Parameter Regularization (weight decay)

- Because H is real and symmetric, we can decompose it into a diagonal matrix Λ and an orthonormal basis of eigenvectors, Q , such that

- $H = Q\Lambda Q^T$

$$\begin{aligned}\tilde{w} &= (Q\Lambda Q^T + \alpha I)^{-1} Q\Lambda Q^T w^* \\ &= [Q(\Lambda + \alpha I)Q^T]^{-1} Q\Lambda Q^T w^* \\ &= Q(\Lambda + \alpha I)^{-1} \Lambda Q^T w^*.\end{aligned}$$

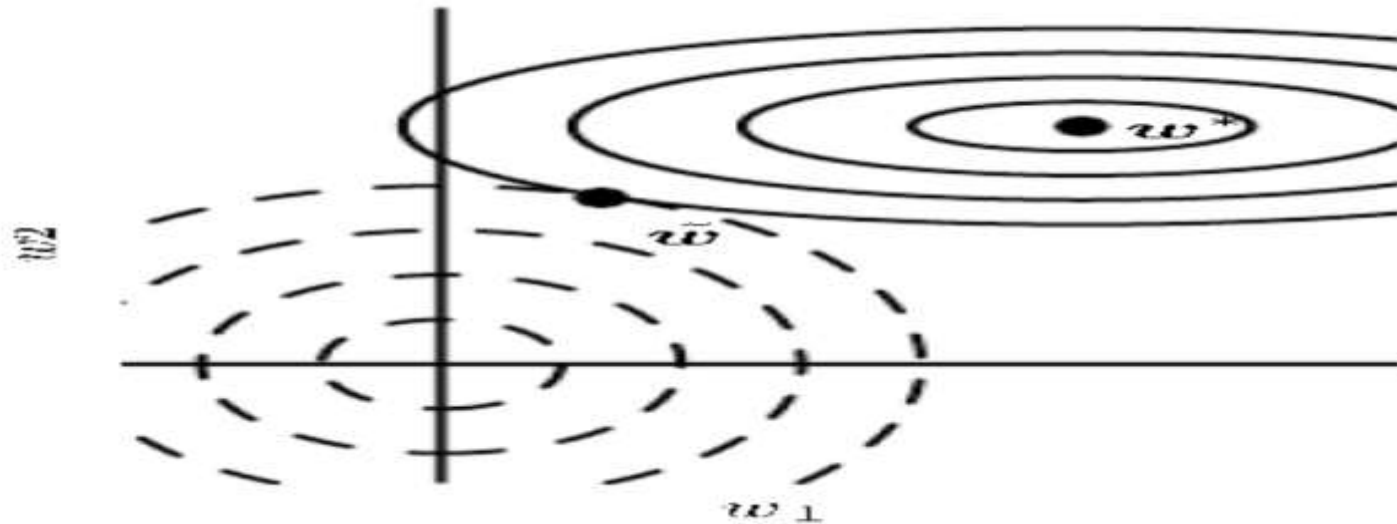
- the effect of weight decay is to rescale w^* along the axes defined by the eigenvectors of H .
- Specifically, the component of w^* that is aligned with the i -th eigenvector of H is rescaled by a factor of $\frac{\lambda_i}{\lambda_i + \alpha}$

L^2 Parameter Regularization (weight decay)

- Specifically, the component of w^* that is aligned with the i -th eigenvector of H is rescaled by a factor of $\frac{\lambda_i}{\lambda_i + \alpha}$
- Along the directions where the eigenvalues of H are **relatively large**,
- for example, where $\lambda_i \gg \alpha$, **the effect of regularization is relatively small.**
- However, components with $\lambda_i \ll \alpha$ will be shrunk to have nearly zero magnitude.

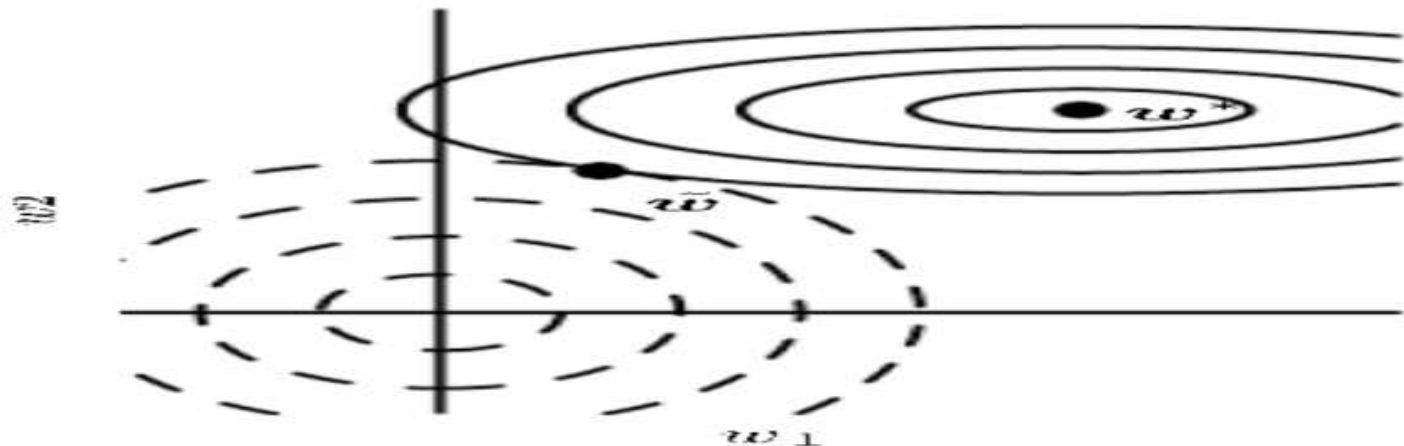
L^2 Parameter Regularization (weight decay)

- An illustration of the effect of L^2 (or weight decay) regularization on the value of the optimal w .
- **solid ellipses represent contours of equal value of unregularized, objective.**
- **dotted circles represent contours** At the point $\tilde{w}$, these competing objectives reach an equilibrium of equal value of the L^2 regularizer.
- .



L^2 Parameter Regularization (weight decay)

- In the first dimension, the eigenvalue of the Hessian of J is small.
- The objective function **does not increase much** when moving horizontally away from w^* .
- the objective function does not express a strong preference along this direction, the regularizer has a strong effect on this axis.
- The regularizer pulls w_1 close to zero.
- if there are directions with small eigenvalues, the objective function is **less sensitive to movements** in those directions, **indicating flatter regions**



L^2 Parameter Regularization (weight decay)

- In the second dimension, the objective function is **very sensitive to movements away from w^*** . the increase in the objective function will be substantial
- The corresponding eigenvalue is large, **indicating high curvature**.
- As a result, weight decay affects the position of w_2 relatively little.
- small changes in the weights can lead to large changes in the loss. This is critical during optimization, as it implies that care must be taken when moving in these directions to avoid overshooting the minimum.

L^2 Parameter Regularization (weight decay)

- For linear regression, the cost function is the sum of squared errors:
- $(Xw - y)^T (Xw - y)$
- When we add L^2 regularization, the objective function changes to
- $(Xw - y)^T (Xw - y) + \frac{1}{2} \alpha w^t w$

$$w = (X^T X)^{-1} X^T y$$

$$w = (X^T X + \alpha I)^{-1} X^T y.$$

- $X^T X$ is proportional to the covariance matrix $\frac{1}{m} X^T X$
- Using $L2$ regularization replaces this matrix with $(X^T X + \alpha I)^{-1}$

L^2 Parameter Regularization (weight decay)

- Using L^2 regularization replaces this matrix with $(X^T X + \alpha I)^{-1}$
- $\tilde{w} = Q(\Lambda + \alpha I)^{-1} \Lambda Q^T w^*$
- Each element of $Q^T w^*$ gets scaled by $\frac{\lambda_i}{\lambda_i + \alpha}$ before it is rotated back by Q
- If $\lambda_i \gg \alpha$ $\frac{\lambda_i}{\lambda_i + \alpha} = 1$
- If $\lambda_i \ll \alpha$ $\frac{\lambda_i}{\lambda_i + \alpha} = 0$
- Thus only significant direction (larger eigen value) will be retained
- Effective parameter = $\frac{\lambda_i}{\lambda_i + \alpha} < n$

L^2 Parameter Regularization (weight decay)

$$\bullet (\Lambda + \alpha I)^{-1} \Lambda = \begin{bmatrix} \frac{1}{\lambda_1 + \alpha} & 0 & 0 & 0 \\ 0 & \frac{1}{\lambda_2 + \alpha} & 0 & 0 \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & \frac{1}{\lambda_n + \alpha} \end{bmatrix} * \begin{bmatrix} \lambda_1 & 0 & 0 & 0 \\ 0 & \lambda_2 & 0 & 0 \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & \lambda_n \end{bmatrix}$$

L^2 Parameter Regularization (weight decay)

$$\bullet = \begin{bmatrix} \frac{\lambda_1}{\lambda_1 + \alpha} & \frac{0}{1} & \frac{0}{1} & \frac{0}{1} \\ \frac{0}{1} & \frac{\lambda_2}{\lambda_2 + \alpha} & \frac{0}{1} & \frac{0}{1} \\ \frac{0}{1} & \frac{0}{1} & \frac{0}{1} & \frac{0}{1} \\ \frac{0}{1} & \frac{0}{1} & \frac{0}{1} & \frac{\lambda_n}{\lambda_n + \alpha} \end{bmatrix}$$

- If $\frac{\lambda_i}{\lambda_i + \alpha} = 1$,
- This means $\lambda_i = \lambda_i + \alpha$
- $\alpha = 0$
- then **w^* weight remain the same & no Change.** the effect of L2 regularization essentially vanishes,
- this implies that under very weak regularization, the model behaves similarly to one without regularization,

L^2 Parameter Regularization (weight decay)

- **If α is very small:**
 - The regularization effect is minimal.
 - As a result, the weight update will primarily reflect the gradient from the loss function, and the weights w^* will not change significantly.
-
- If $\frac{\lambda_i}{\lambda_i + \alpha} = 0$,
 - This means $\lambda_i = 0$
 - it means that there is no regularization applied to the corresponding weight. Therefore, the weight will not be penalized during the optimization process.
 - the L2 regularization term does not influence the weight w^*
 - The weight w^* will be updated according to the standard optimization process (e.g., gradient descent) without any shrinkage or penalty from regularization.

L^2 Parameter Regularization (weight decay)

- When $0 < \frac{\lambda_i}{\lambda_i + \alpha} < 1$
- L2 regularization is active, but not dominant.
- The weights w^* will be smaller than they would be without regularization, but they will still retain some magnitude.
- This leads to a balance between fitting the training data and controlling for overfitting, helping improve generalization on unseen data.

L^2 Parameter Regularization (weight decay)

- L^2 regularization replaces this matrix with $(X^T X + \alpha I)^{-1}$
- We can see that L^2 regularization causes the learning algorithm to “perceive” the input X as having higher variance, which makes it shrink the weights on features whose covariance with the output target is low compared to this added variance.
- **Effect on Variance:**
- **Increased Perceived Variance:** The L^2 regularization term effectively adds a constant "noise" to the system.
- input X as having higher variance because the model is now considering not only the data but also this additional penalty on the weights.
- As a result, features that have low covariance with the target variable (meaning they have less influence on predicting the output) will see their weights shrink more significantly compared to features that have a higher covariance.

L^2 Parameter Regularization (weight decay)

- **Weight Shrinkage:**
- **Covariance with Output:** If a feature x_i has low covariance with the output y , the model perceives that its contribution is **less significant**,
 - especially in the presence of the added variance due to regularization.
- The gradient descent updates for these weights will be smaller because the effective "error" (including the regularization term) leads to a stronger penalty.
- This causes weights corresponding to less **important features to shrink toward zero** more aggressively.

Regularization techniques to avoid overfitting

- If you observe that your neural network is overfitting the training data, your network might be too complex and need to be simplified.
- ***L2 regularization***
- The basic idea of L2 regularization is that it penalizes the error function by adding a *regularization term* to it.
- This, in turn, reduces the weight values of the hidden units and
- makes them too small, very close to zero, to help simplify the model.

Regularization techniques to avoid overfitting

- First, we update the error function by adding the regularization term:

$$\text{error function}_{\text{new}} = \text{error function}_{\text{old}} + \text{regularization term}$$

$$\text{L2 regularization term} = \frac{\lambda}{2m} \times \sum \|w\|^2$$

- where lambda (λ) is the regularization parameter, m is the number of instances, and w is the weight.

$$\text{error function}_{\text{new}} = \text{error function}_{\text{old}} + \frac{\lambda}{2m} \times \sum \|w\|^2$$

Regularization techniques to avoid overfitting

- L2 regularization is also known as *weight decay*, as it forces the
- weights to decay toward zero (but not exactly zero).

The diagram shows the weight decay formula with labels and arrows pointing to its components:

$$W_{new} = W_{old} - \alpha \left(\frac{\partial Error}{\partial W_x} \right)$$

Labels and arrows:

- Old weight**: points to W_{old}
- Derivative of error with respect to weight**: points to $\frac{\partial Error}{\partial W_x}$
- New weight**: points to W_{new}
- Learning rate**: points to α

L2 Regularization

- The regularized cost function is:
- $J(\theta) = Loss(\theta) + \lambda \|\theta\|^2$
- Where $Loss(\theta)$ is cost function (e.g., mean squared error),
- $\|\theta\|^2 = \sum_i \theta_i^2$ is the L2 norm (sum of squared weights)
- λ is the regularization strength (a hyperparameter).
- **Gradient of L2 Regularization:**
- $\nabla_{\theta}(\lambda \|\theta\|^2) = 2 \lambda \theta$
- $\nabla_{\theta} J(\theta) = \nabla_{\theta} Loss(\theta) + 2 \lambda \theta$

L2 Regularization

- **Weight Update Rule:**
- In gradient descent, the weight update rule is:
- $\theta_{new} = \theta_{old} - \alpha \nabla_{\theta} J(\theta)$
- Where α is the learning rate
- $\theta_{new} = \theta_{old} - \alpha (\nabla_{\theta} Loss(\theta) + 2 \lambda \theta_{old})$

L2 Regularization

- $\theta_{new} = \theta_{old} - \alpha (\nabla_{\theta} Loss(\theta) + 2 \lambda \theta_{old})$
- when the **regularization term is large**.
- **If $\alpha \times 2 \lambda$ is large to cancel out θ_{old} , the update**
- $\theta_{new} = \theta_{old} - \alpha \times 2 \lambda \theta_{old}$
- $= \theta_{old} (1 - 2 \alpha \lambda)$
- When the New Weight Becomes Zero:
- **$1 - 2 \alpha \lambda = 0$, this would imply $\theta_{new} = 0$**
- **In other words, the weight θ becomes 0 when $2 \alpha \lambda = 1$**
- **Or $\lambda = \frac{1}{2 \alpha}$**
- This means that if the regularization strength λ is large enough such that it satisfies this condition, the weight θ_{new} will be zero after the update.

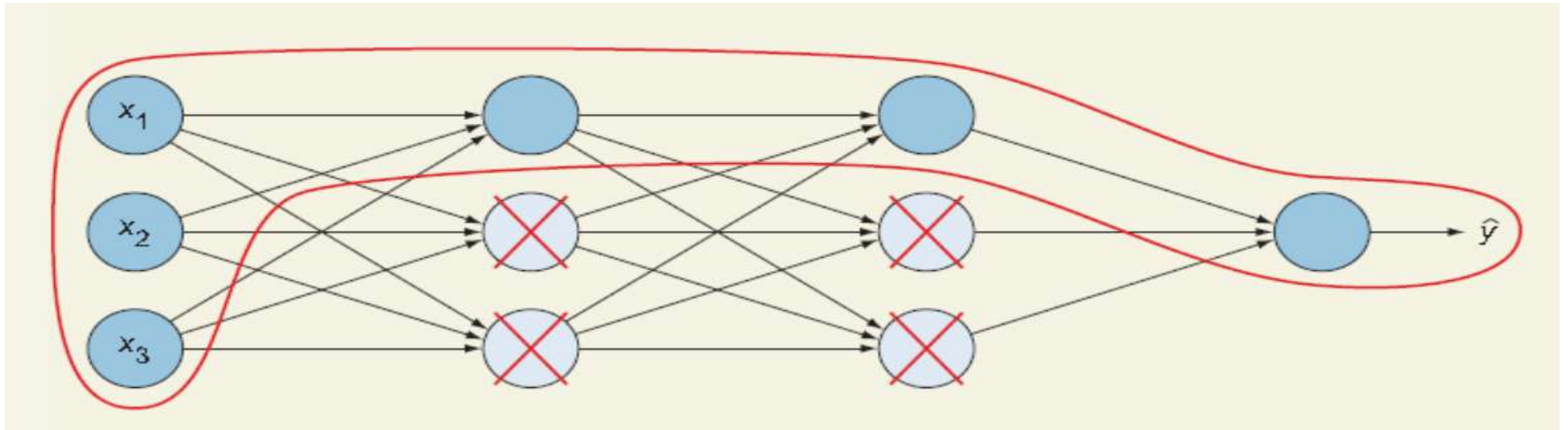
L2 Regularization

- If the regularization term is **too large** (specifically if λ is large enough relative to the learning rate α), the weight update can "shrink" the weight completely to zero.
- This happens because the L2 regularization term acts like a penalty that pushes the weights toward zero,
- and if λ is too large, it can overwhelm the gradient from the loss function itself, leading to a weight update that nullifies the weight.

Regularization techniques to avoid overfitting

- Reducing weights leads to a simpler neural network
- To see how this works, consider: **if the regularization term is so large** that, when multiplied with the learning rate, it will be equal to W old,
- then this will make the **new weight equal to zero**.
- This cancels the effect of this neuron, leading to a simpler neural network with fewer neurons.
- In practice, L2 regularization does not make the weights equal to zero.
- It just makes them smaller to reduce their effect.
- A large regularization parameter (λ) lead to negligible weights.
- When weights are negligible, model will not learn much from these units.
- This will make the network simpler and thus reduce overfitting

Regularization techniques to avoid overfitting



L2 regularization reduces the weights and simplifies the network to reduce overfitting

```
model.add(Dense(units=16, kernel_regularizer=regularizers.l2( $\lambda$ ), activation='relu'))
```

Dataset Augmentation

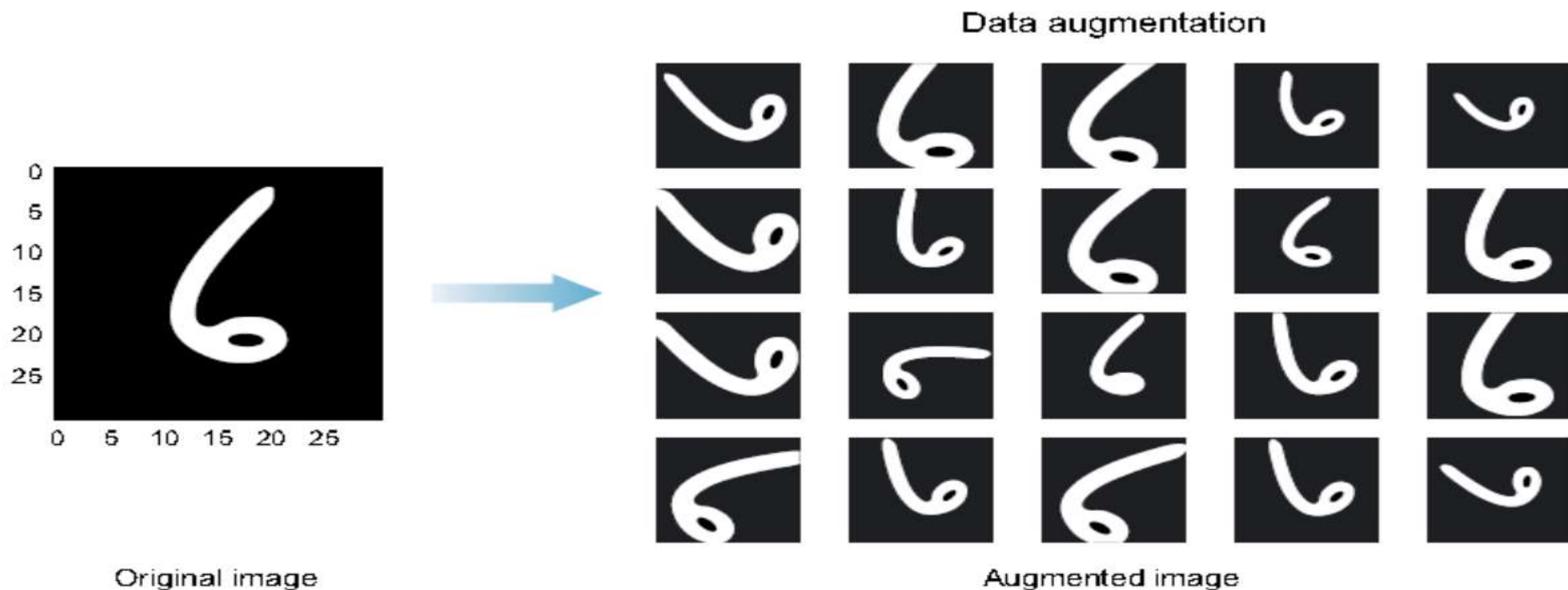
- The best way to make a machine learning model generalize better is to **train it on more data**.
- in practice, the amount of data we have is limited.
- One way to get around this problem is to **create fake data** and add it to the training set.
- This approach is easiest for classification. A classifier needs to take a complicated, high dimensional input x and summarize it with a single category identity y .
- This means that the main task facing a classifier is to be **invariant** to a wide variety of transformations.
- We can generate new (x, y) pairs easily just by transforming the x inputs in our training set.

Dataset Augmentation

- One way to **avoid overfitting is to obtain more data.**
- Since this is not always a feasible option, we can augment our training data by generating **new instances of the same images with some transformations.**
- Data augmentation can be an inexpensive way to give your learning algorithm more training data and therefore **reduce overfitting.**
- The many image-augmentation techniques **include flipping, rotation, scaling, zooming, lighting conditions, and many other transformations** that you can apply to your dataset to provide a variety of images to train on.

Dataset Augmentation

- Various image augmentation techniques applied to image of digit 6



Dataset Augmentation

- Data augmentation is considered a regularization technique because allowing the network to see many variants of the object reduces its dependence on the original form of the object during feature learning.
- This makes the network more resilient when tested on new data

Dataset Augmentation

Data augmentation in Keras looks like this:

```
from keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(horizontal_flip=True, vertical_flip=True)

datagen.fit(training_set)
```

Imports ImageDataGenerator from Keras

Generates batches of new image data. ImageDataGenerator takes transformation types as arguments. Here, we set horizontal and vertical flip to True. See the Keras documentation (or your DL library) for more transformation arguments.

Computes the data augmentation on the training set

Dataset Augmentation

- Dataset augmentation is effective for speech recognition tasks.
- Injecting noise in the input to a neural network
- For many classification and even some regression tasks,
- the task should still be possible to solve even if small random noise is added to the input.
- to improve the robustness of neural networks is simply to train them with random noise applied to their inputs.
- Input noise injection is part of some unsupervised learning algorithms such as the denoising autoencoder

Noise Robustness

- **Types of Noise:**
- **Input Noise:** Random variations in input data (e.g., sensor inaccuracies, image noise).
- **Weight Noise:** Variations in model weights due to quantization, pruning, or other perturbations.
- **Label Noise:** Incorrect labels in the training dataset
- **use of noise applied to the inputs as a dataset augmentation strategy.**
- noise injection can be much more powerful than simply shrinking the parameters, especially when the noise is added to the hidden units.
- Another way that noise has been used in the service of regularizing models is by adding it to the weights.

This technique has been used primarily in the context of **recurrent neural networks**

Noise Robustness

- **Noise applied to the weights** can also be interpreted as equivalent to a more **traditional form of regularization**,
- Consider the regression setting

$$J = \mathbb{E}_{p(\mathbf{x}, y)} \left[(\hat{y}(\mathbf{x}) - y)^2 \right] . \quad ($$

The training set consists of m labeled examples $\{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(m)}, y^{(m)})$

- By adding noise to the weights during training, the model learns to maintain performance even when there are small variations in the weight values.
- leading to improved performance on unseen data.

Noise Robustness

- We now assume that with each input presentation we also include a random perturbation $\epsilon_W \sim \mathcal{N}(\epsilon; \mathbf{0}, \eta \mathbf{I})$ of network weights.
- Let us imagine that we have a standard l -layer MLP.
- We denote the perturbed model as $\hat{y}_{\epsilon_W}(\mathbf{x})$.
- The objective function

$$\begin{aligned}\tilde{J}_W &= \mathbb{E}_{p(\mathbf{x}, y, \epsilon_W)} \left[(\hat{y}_{\epsilon_W}(\mathbf{x}) - y)^2 \right] \\ &= \mathbb{E}_{p(\mathbf{x}, y, \epsilon_W)} \left[\hat{y}_{\epsilon_W}^2(\mathbf{x}) - 2y\hat{y}_{\epsilon_W}(\mathbf{x}) + y^2 \right]\end{aligned}$$

Noise Robustness

- For small η , the minimization of J with added weight noise (with covariance ηI) is equivalent to **minimization of J with an additional regularization term**

$$\eta \mathbb{E}_{p(x,y)} [\|\nabla_{\mathbf{w}} \hat{y}(x)\|^2]$$

- As the model converges to a minimum, it may find points where small perturbations in weights result in little to no change in the loss.
- model is not overly sensitive to noise in the data.
- finding points that are not merely minima, but minima surrounded by flat regions

Noise Robustness

- In the simplified case of linear regression ,
- $\hat{y}(x) = W^T X + b$, this regularization term collapses into

$$\eta \mathbb{E}_{p(x)} [\|x\|^2]$$

- , which is not a function of parameters and therefore does not
- contribute to the gradient of $J \sim W$ with respect to the model parameters

Adding noise to the input is equivalent to weight decay L2 Regularization

- $\tilde{x}_i = x_i + \xi$
- Predicted $\hat{y} = \sum_{i=1}^n w_i * x_i$
- $\tilde{y} = \sum_{i=1}^n w_i * \tilde{x}_i$
- $= \sum_{i=1}^n w_i * x_i + \sum_{i=1}^n w_i * \xi_i$
- $= \hat{y} + \sum_{i=1}^n w_i * \xi_i$

Adding noise to the input is equivalent to weight decay L2 Regularization

- True Output = $E[(\tilde{y} - y)^2] = E[(\hat{y} + \sum_{i=1}^n w_i * x_i - y)^2]$
- $= E[(\hat{y} - y) + \sum_{i=1}^n w_i * x_i]^2$
- $= E(\hat{y} - y)^2 + E(2(\hat{y} - y) \sum_{i=1}^n w_i * \xi_i) + E(\sum_{i=1}^n w_i * \xi_i)^2$
- Since (independent) $E(2(\hat{y} - y) \sum_{i=1}^n w_i * \xi_i) = 0$
- $= E(\hat{y} - y)^2 + 0 + E(\sum_{i=1}^n w_i * \xi_i)^2$
- Since $E(\xi_i^2) = \sigma^2$ variance
- $= E(\hat{y} - y)^2 + \sigma \sum_{i=1}^n w_i^2$
- $\sigma \sum_{i=1}^n w_i^2$ L2 Norm Form

Example

Dataset: Imagine we have a small dataset with two features X_1 and X_2 and a target variable y :

X_1	X_2	y
1.0	200.0	5.0
2.0	210.0	5.5
3.0	220.0	6.0
4.0	230.0	6.5
5.0	240.0	7.0

Example

Step 1: Fit a Linear Model Without Regularization

1. Model:

$$\hat{y} = W_1 X_1 + W_2 X_2 + b$$

2. Cost Function (without regularization):

$$J(W_1, W_2, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (W_1 X_{1i} + W_2 X_{2i} + b))^2$$

3. **Fitting the Model:** Without regularization, you might find that W_1 is relatively small (due to low variability in X_1), while W_2 becomes larger because X_2 has higher values. This could lead to overfitting if the model captures noise.

Example

Step 2: Fit a Linear Model With L2 Regularization

1. Regularized Cost Function:

$$J(W_1, W_2, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (W_1 X_{1i} + W_2 X_{2i} + b))^2 + \frac{\lambda}{2} (W_1^2 + W_2^2)$$

2. Impact of Regularization:

- **Penalty for Large Weights:** The term $\frac{\lambda}{2} (W_1^2 + W_2^2)$ discourages large weights. If λ is set to a reasonable value, this helps stabilize W_1 and W_2 .

Example

Step 3: Expected Value of Features

- Calculate the expected values:
 - For X_1 :

$$E_p(x)[X_1^2] = \frac{1}{5}(1^2 + 2^2 + 3^2 + 4^2 + 5^2) = \frac{1}{5}(55) = 11$$

- For X_2 :

$$E_p(x)[X_2^2] = \frac{1}{5}(200^2 + 210^2 + 220^2 + 230^2 + 240^2) = \frac{1}{5}(10000 + 44100 + 48400 + 52900 + 57600) \approx 42600$$

Example

- **Step 4: Comparing Models**

- **Without Regularization:**

- Model might produce larger weights for W_2 (e.g., $W_1=0.1$, $W_2=0.3$) due to the scale of X_2 .

- **With Regularization:**

- Regularization shrinks W_2 (e.g., $W_1=0.1$, $W_2=0.1$), leading to a more stable model that generalizes better to unseen data.

estimate weights with L2 regularization (Ridge regression),

Step 1: Formulate the Design Matrix

First, we need to create the design matrix $\mathbf{X}$ and the target vector $\mathbf{y}$.

$$\mathbf{X} = \begin{bmatrix} 1 & 1.0 & 200.0 \\ 1 & 2.0 & 210.0 \\ 1 & 3.0 & 220.0 \\ 1 & 4.0 & 230.0 \\ 1 & 5.0 & 240.0 \end{bmatrix}$$

Here, the first column represents the bias term (intercept).

estimate weights with L2 regularization (Ridge regression),

Step 2: Set Up the Regularization Parameter

Let's choose a regularization parameter λ . For this example, we'll use $\lambda = 0.1$.

Step 3: Calculate the Weights Using the Normal Equation

The weights in Ridge regression can be calculated using the following formula:

$$\mathbf{W} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$$

where $\mathbf{I}$ is the identity matrix.

1. Calculate $\mathbf{X}^T \mathbf{X}$:

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 5 & 15 & 1150 \\ 15 & 55 & 11500 \\ 1150 & 11500 & 2470000 \end{bmatrix}$$

estimate weights with L2 regularization (Ridge regression),

2. Add $\lambda \mathbf{I}$:

$$\lambda \mathbf{I} = 0.1 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.1 & 0 & 0 \\ 0 & 0.1 & 0 \\ 0 & 0 & 0.1 \end{bmatrix}$$

3. Combine the matrices:

$$\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I} = \begin{bmatrix} 5.1 & 15 & 1150 \\ 15 & 55.1 & 11500 \\ 1150 & 11500 & 2470000.1 \end{bmatrix}$$

estimate weights with L2 regularization (Ridge regression),

4. Calculate $\mathbf{X}^T \mathbf{y}$:

$$\mathbf{X}^T \mathbf{y} = \begin{bmatrix} 25.0 \\ 82.5 \\ 17150.0 \end{bmatrix}$$

Step 4: Compute the Weights

1. Calculate the inverse:

Using a matrix calculator or numerical software, calculate:

$$(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1}$$

estimate weights with L2 regularization (Ridge regression),

2. Multiply by $\mathbf{X}^T \mathbf{y}$ to find $\mathbf{W}$:

Putting it all together, you can compute the weights $\mathbf{W}$. For this example, let's say you compute:

$$\mathbf{W} \approx \begin{bmatrix} 4.5 \\ 0.07 \\ 0.02 \end{bmatrix}$$

Interpreting the Weights

- Bias Term: $W_0 \approx 4.5$ indicates the model's prediction when both features are zero.
- Feature Weights:
 - $W_1 \approx 0.07$ means that for every unit increase in X_1 , the output y increases by 0.07.
 - $W_2 \approx 0.02$ means that for every unit increase in X_2 , the output y increases by 0.02.

estimate weights without Regularization

$$\mathbf{W} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

1. Calculate $\mathbf{X}^T \mathbf{X}$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 5 & 15 & 1150 \\ 15 & 55 & 11500 \\ 1150 & 11500 & 2470000 \end{bmatrix}$$

2. Calculate $\mathbf{X}^T \mathbf{y}$

$$\mathbf{X}^T \mathbf{y} = \begin{bmatrix} 25.0 \\ 82.5 \\ 17150.0 \end{bmatrix}$$

3. Calculate the Inverse of $\mathbf{X}^T \mathbf{X}$

Using a calculator or numerical software, compute the inverse of $\mathbf{X}^T \mathbf{X}$.

4. Multiply by $\mathbf{X}^T \mathbf{y}$

After finding the inverse, multiply it by $\mathbf{X}^T \mathbf{y}$ to find $\mathbf{W}$.

estimate weights without Regularization

$$\mathbf{W} \approx \begin{bmatrix} 0.5 \\ 0.02 \\ 0.03 \end{bmatrix}$$

Interpreting the Weights

- Bias Term: $W_0 \approx 0.5$ indicates the model's prediction when both features are zero.
- Feature Weights:
 - $W_1 \approx 0.02$ means that for every unit increase in X_1 , the output y increases by 0.02.
 - $W_2 \approx 0.03$ means that for every unit increase in X_2 , the output y increases by 0.03.

Early Stopping

- A training iteration, or *epoch*, is when the model goes a full cycle and sees the entire training dataset at once.
- The epoch hyperparameter is set to define how many iterations our network continues training.
- **The more training iterations, the more our model learns the features of our training data.**
- To diagnose whether your network needs more or fewer training epochs, **keep your eyes on the training and validation error values.**

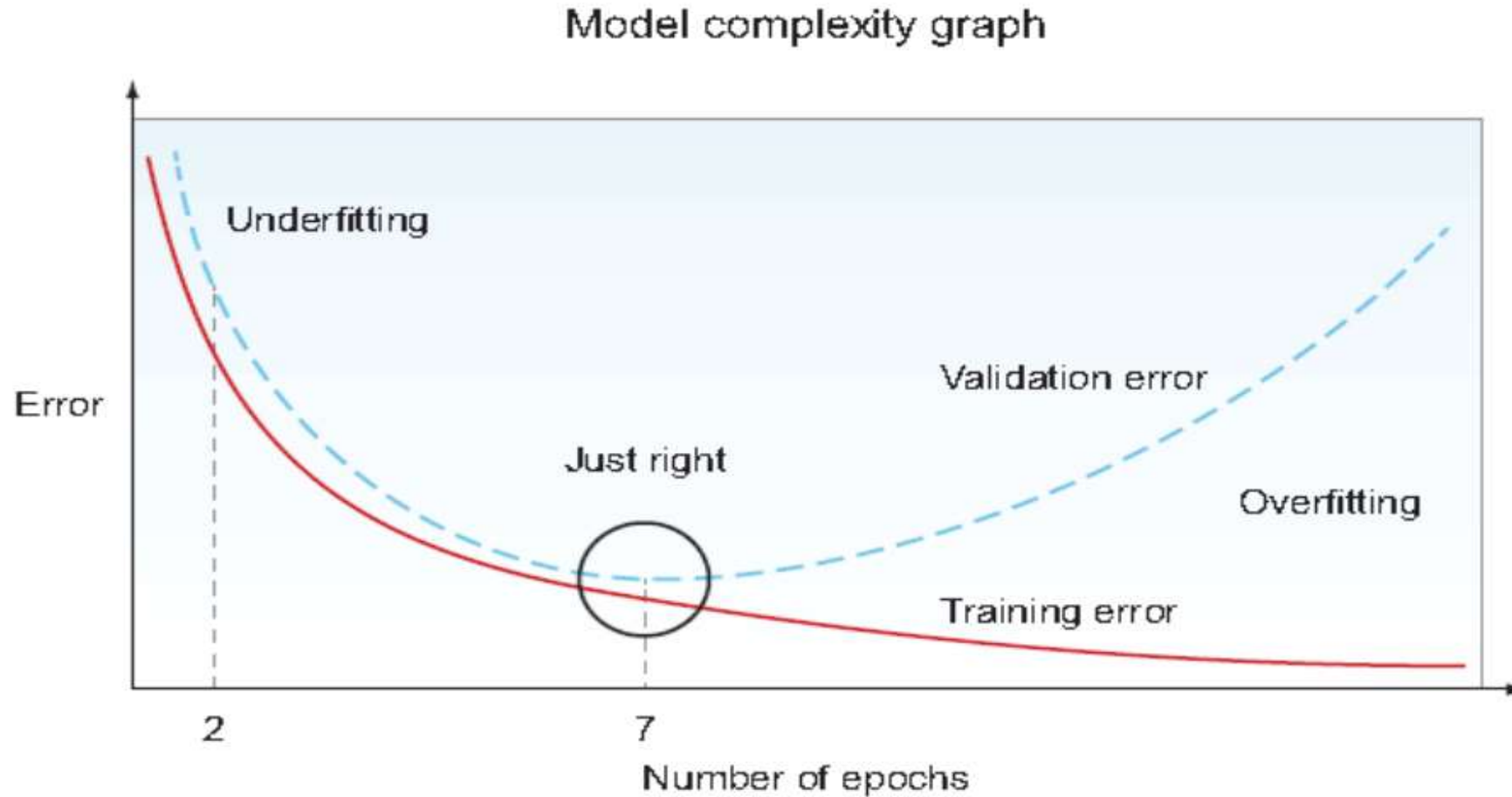
Early Stopping

- The intuitive way to think about this is that we want to continue training as long as the error value is decreasing. Correct
- Epoch 1, Training Error: 5.4353, Validation Error: 5.6394
- Epoch 2, Training Error: 5.1364, Validation Error: 5.2216
- Epoch 3, Training Error: 4.7343, Validation Error: 4.8337
- You can see that both training and validation errors are decreasing. This means the network is still learning.
- It doesn't make sense to stop the training at this point.
- The network is clearly still making progress toward the minimum error. Let's let it train for six more epochs and observe the results

Early Stopping

- Epoch 6, Training Error: 3.7312, Validation Error: 3.8324
- Epoch 7, Training Error: 3.5324, Validation Error: 3.7215
- **Epoch 8, Training Error: 3.7343, Validation Error: 3.8337**
- The training error is still improving, but the validation error started oscillating from epoch 8 onward.
- Improving train_error while not improving val_error means the network is starting **to overfit the training data** and failing to generalize to the validation data.

Early Stopping



Improving **train_error** while not improving **val_error** means the network is starting to overfit.

Early Stopping

- We need to find a way to stop the training just before it starts to overfit. **This technique is called *early stopping*.**
- Early stopping is an algorithm widely used to determine the right time to stop the training process before overfitting happens.
- It simply monitors the validation error value and stops the training when the value starts to increase.
- Here is the early stopping function in Keras:
- `EarlyStopping(monitor='val_loss', min_delta=0, patience=20)`

Early Stopping

- **EarlyStopping(monitor='val_loss', min_delta=0, patience=20)**
- **monitor**—The metric you monitor during training. Usually we want to **keep an eye on val_loss** because it represents our internal testing of model performance.
- If the network is doing well on the validation data, it will probably do well on test data and production

min_delta—The minimum change that qualifies as an improvement

Early Stopping

- **EarlyStopping(monitor='val_loss', min_delta=0, patience=20)**
- patience—This variable tells the algorithm how many epochs it should wait before stopping the training if the error does not improve.
- For example, if we set patience equal to 1, the training will stop at the epoch where the error increases.

We must be a little flexible, though, because it is very common for the error to oscillate a little and continue improving.

We can stop the training if it hasn't improved in the last 10 or 20 epochs

Early Stopping

- algorithm terminates when no parameters have improved over the best recorded validation error for some pre-specified number of iterations.
- Let n be the number of steps between evaluations.
- Let p be the “patience,” the number of times to observe worsening validation set error before giving up.

Early Stopping

Let θ_o be the initial parameters.

$$\theta \leftarrow \theta_o$$

$$i \leftarrow 0$$

$$j \leftarrow 0$$

$$v \leftarrow \infty$$

$$\theta^* \leftarrow \theta$$

$$i^* \leftarrow i$$

Early Stopping

```
while  $j < p$  do  
    Update  $\theta$  by running the training algorithm for  $n$  steps.  
     $i \leftarrow i + n$   
     $v' \leftarrow \text{ValidationSetError}(\theta)$   
    if  $v' < v$  then  
         $j \leftarrow 0$   
         $\theta^* \leftarrow \theta$   
         $i^* \leftarrow i$   
         $v \leftarrow v'$   
    else  
         $j \leftarrow j + 1$   
    end if  
end while  
Best parameters are  $\theta^*$ , best number of training steps is  $i^*$ 
```

Early Stopping

- A meta-algorithm for using early stopping to determine how long to train, then retraining on all the data.

Let $\mathbf{X}^{(\text{train})}$ and $\mathbf{y}^{(\text{train})}$ be the training set.

Split $\mathbf{X}^{(\text{train})}$ and $\mathbf{y}^{(\text{train})}$ into $(\mathbf{X}^{(\text{subtrain})}, \mathbf{X}^{(\text{valid})})$ and $(\mathbf{y}^{(\text{subtrain})}, \mathbf{y}^{(\text{valid})})$ respectively.

Run early stopping (algorithm 7.1) starting from random θ using $\mathbf{X}^{(\text{subtrain})}$ and $\mathbf{y}^{(\text{subtrain})}$ for training data and $\mathbf{X}^{(\text{valid})}$ and $\mathbf{y}^{(\text{valid})}$ for validation data. This returns i^* , the optimal number of steps.

Set θ to random values again.

Train on $\mathbf{X}^{(\text{train})}$ and $\mathbf{y}^{(\text{train})}$ for i^* steps.

Early Stopping

- This strategy avoids the high cost of retraining the model from scratch
- Early stopping is also useful because it reduces the computational cost of the training procedure.
- it also has the benefit of providing regularization without requiring the addition of penalty terms to the cost function or the computation of the gradients of such additional terms

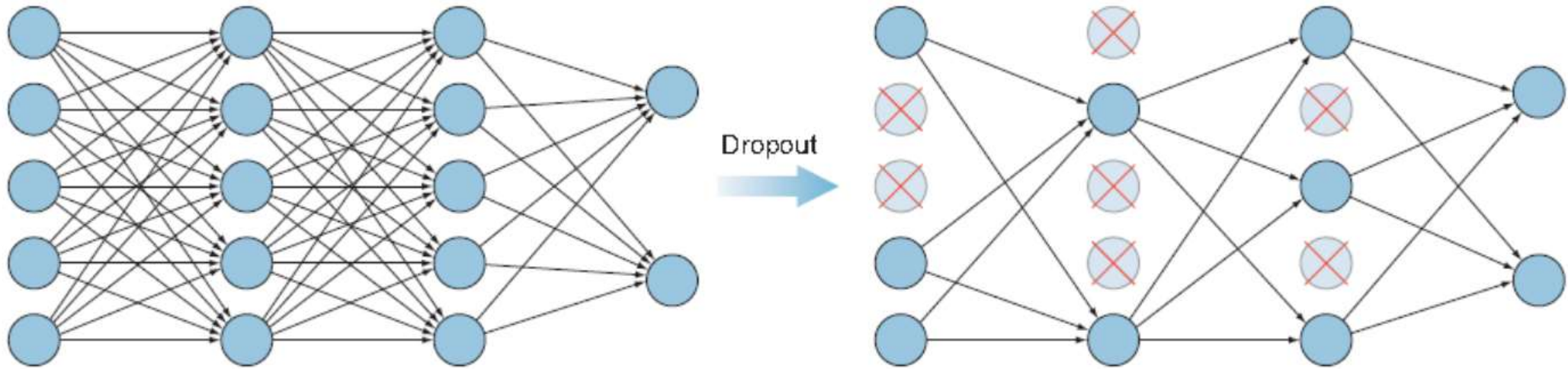
Dropout layers

- **Dropout** is another **regularization technique**.
- very effective **for simplifying a neural network and avoiding overfitting**.
- at every training iteration, every neuron has a probability p of being temporarily ignored (dropped out) during this training iteration.
- This means it may be active during subsequent iterations.
- to intentionally pause the learning on some of the network neurons, it is quite surprising how well this technique works.
- The probability p is a hyperparameter that is called *dropout rate*
- and is typically set in the range of 0.3 to 0.5.
- Start with 0.3, and if you see signs of overfitting, increase the rate.

Dropout layers

- Both L2 regularization and dropout aim to reduce network complexity by reducing its neurons' effectiveness.
- The difference is that **dropout completely *cancels*** the effect of some neurons with every iteration,
- **while L2 regularization just reduces the weight values** to *reduce* the neurons' effectiveness.
- Both lead to a more robust, resilient neural network and reduce overfitting.
- It is recommended that you use both types of regularization techniques in your network

Dropout layers



Dropout turns off a percentage of the neurons that make up a network layer

Why do we need dropout layers?

- Neurons develop codependency among each other during training, which controls the individual power of each neuron, leading to overfitting of training data.
- Each neuron represents a certain feature that, when multiplied by a weight, is transformed into another feature.
- When we randomly turn off some of these nodes, we force the other nodes to learn patterns without relying on only one or two features, because any feature can be randomly dropped out at any point.
- This results in spreading out the weights among all the features, leading to more trained neurons

Why do we need dropout layers?

- Dropout helps reduce interdependent learning among the neurons.
- In ensemble learning, we train a **number of weaker classifiers** separately, and then we use them at test time by averaging the responses of all ensemble members.
- Since each classifier has been trained separately, it has learned different aspects of the data, and their mistakes (errors) are different. Combining them helps to produce a stronger classifier, which is less prone to overfitting.

Where does the dropout layer go in the CNN architecture?

- CNN consists of alternating **convolutional and pooling layers, ending with fully connected layers.**
- To prevent overfitting, it's become standard practice after **you flatten the image** to inject a few dropout layers.
- between the fully connected layers at the end of the architecture. Why?
- Because dropout is known to work well in the fully connected layers of convolutional neural nets.
- Its effect in convolutional and pooling layers is, however, not well studied yet:
- CNN architecture: ... CONV $\Rightarrow$ POOL $\Rightarrow$ Flatten $\Rightarrow$ DO $\Rightarrow$ FC $\Rightarrow$ DO $\Rightarrow$ FC

Dropout

```
# CNN and POOL layers  
# ...  
# ...
```

```
model.add(Flatten())
```

Flatten layer

```
model.add(Dropout(rate=0.3))
```

**Dropout layer with
30% probability**

```
model.add(Dense(64, activation='relu'))
```

**FC_1: fully connected
to get all relevant data**

```
model.add(Dropout(rate=0.5))
```

**Dropout layer with
50% probability**

```
model.add(Dense(10, activation='softmax'))
```

**FC_2: outputs a softmax to squash
the matrix into output probabilities
for the 10 classes**

```
model.summary()
```

**Prints the model
architecture summary**

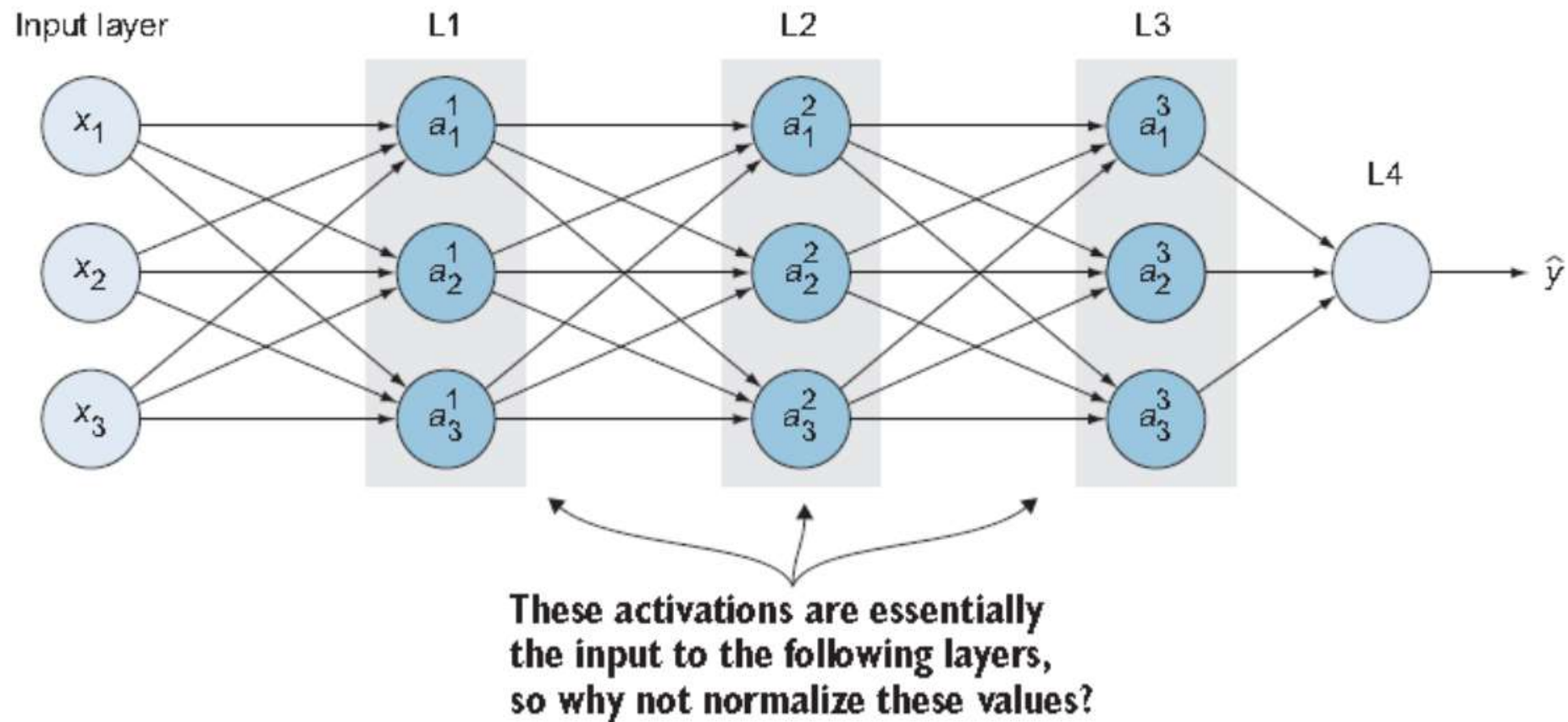
Dropout

- For example, if we set rate to 0.3, it means 30% of the neurons in this layer will be randomly dropped in each epoch.
- So if we have 10 nodes in a layer, 3 of these neurons will be turned off, and 7 will be trained.
- The three neurons are randomly selected, and in the next epoch other randomly selected neurons are turned off, and so on

Batch normalization

- Use of data normalization to speed up learning.
- The normalization techniques were focused on preprocessing the training set before feeding it to the **input layer**.
- If the input layer benefits from normalization, why not do the same thing for the *extracted features* in the hidden units,
- which are changing all the time and get much more improvement in training speed and network resilience
- **This process is called *batch normalization* (BN).**

Batch normalization

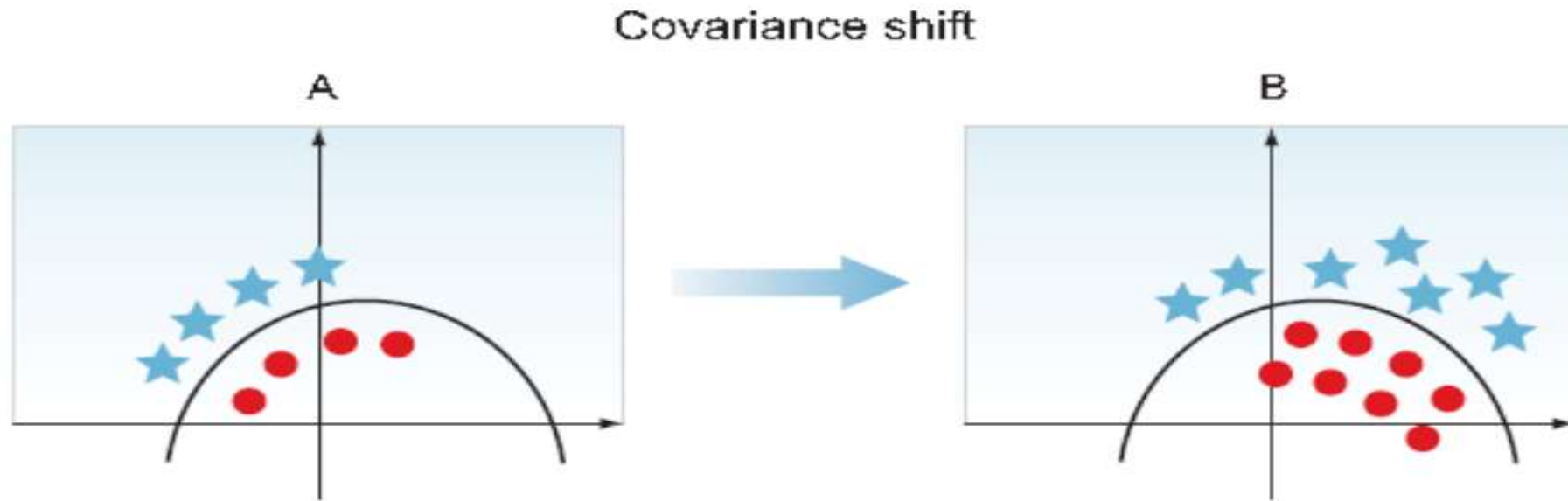


Batch normalization is normalizing the extracted features in hidden units.

The covariate shift problem

- Suppose you are building a cat classifier, and you train your algorithm on images of white cats only.
- When you test this classifier on images with cats that are different colors, it will not perform well. Why?
- Because the model has been trained on a training set with a specific distribution (white cats).
- When the distribution changes in the test set, it confuses the model

The covariate shift problem



Graph A is the training set of only white cats, and graph B is the testing set with cats of various colors.

The circles represent the cat images, and the stars represent the non-cat images.

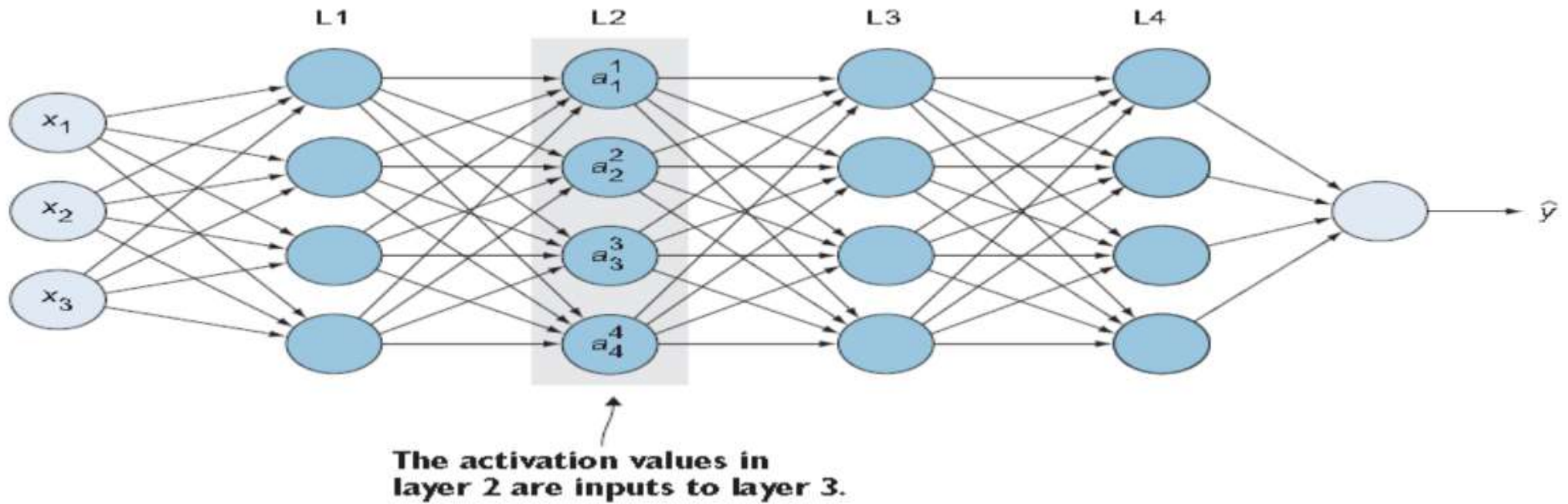
If a model is learning to map dataset X to label y , then if the distribution of X changes, it's known as **covariate shift**.

When that happens, you might need to retrain your learning algorithm.

Covariate shift in neural networks

- Let's look at the network from the third-layer (L3) perspective.

Its input are the activation values in L2 (a_1^2 , a_2^2 , a_3^2 , and a_4^2) which are features extracted from the previous layers.



A simple four-layer MLP. L1 features are input to the L2 layer. The same is true for layers 2, 3, and 4.

Covariate shift in neural networks

- L3 is trying to map these inputs to $\hat{y}$ to make it as close as possible to the label y .
- While the third layer is doing that, the network is adapting the values of the parameters from previous layers.
- As the parameters (w, b) are changing in layer 1, the activation values in the second layer are changing, too.
- So from the perspective of the third hidden layer, the values of the second hidden layer are changing all the time.
- **the MLP is suffering from the problem of covariate shift.**
- **Batch norm reduces the degree of change** in the distribution of the hidden unit values, causing these values to become more stable so that the later layers of the neural network have firmer ground to stand on

Covariate shift in neural networks

- It is important to realize that batch normalization does not cancel or reduce the change in the hidden unit values.

What it does is ensure that the distribution of that change remains the same:

- even if the exact values of the units change, **the mean and variance do not change.**

How does batch normalization work?

- Batch Normalization: **Accelerating Deep Network Training by Reducing Internal Covariate Shift**
- Batch normalization adds an operation in the neural network just before the activation function of each layer to do the following:
 - 1 Zero-center the inputs
 - 2 Normalize the zero-centered inputs
 - 3 Scale and shift the results
- This operation lets the model learn the optimal scale and mean of the inputs for each layer.

How does batch normalization work?

- How the math works in batch normalization
- 1 To zero-center the inputs, the algorithm needs to calculate the input mean and standard deviation
- (the input here means the current mini-batch: hence the term *batch normalization*):

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \leftarrow \text{Mini-batch mean}$$
$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad \leftarrow \text{Mini-batch variance}$$

2 Normalize the input:

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

How does batch normalization work?

- where $\hat{x}$ is the zero-centered and normalized input. Note that there is a variable here that we added (ϵ).
- This is a tiny number (10^{-5}) to avoid division by zero if σ is zero in some estimates.
- **Scale and shift the results.** We multiply the normalized output by a variable γ to scale it and add (β) to shift it
- $y_i \leftarrow \gamma X_i + \beta$
- where y_i is the output of the BN operation, scaled and shifted.

Bagging (ensemble methods)

- Bagging (short for bootstrap aggregating) is a technique **for reducing generalization error by combining several models.**
- The idea is to train several different models separately, then have all of the models vote on the output for test examples.
- This is an example of a general strategy in machine learning called model averaging.
- Techniques employing this strategy are known as **ensemble methods.**

Greedy Layer-Wise Unsupervised Pretraining

- first time to train a deep supervised network without requiring architectural specializations like convolution or recurrence.
- We call this procedure **unsupervised pretraining, or more precisely, greedy layerwise unsupervised pretraining**
- unsupervised learning, trying to capture the shape of the input distribution) can sometimes be useful for another task (supervised learning with the same input domain.
- Greedy layer-wise unsupervised pretraining relies on a single-layer representation learning algorithm.
- such as an RBM, a single-layer autoencoder, a sparse coding model, or another model that learns latent representations

Greedy Layer-Wise Unsupervised Pretraining

- Each layer is pretrained using unsupervised learning, taking the output of the previous layer and producing as output a new representation of the data, whose distribution (or its relation to other variables such as categories to predict) is hopefully simpler.
- used to sidestep the difficulty of jointly training the layers of a deep neural net for a supervised task.
- greedy learning procedure could be used to find a good initialization for a joint learning procedure over all the layers,
- and that this approach could be used to successfully train even fully connected architectures

Greedy Layer-Wise Unsupervised Pretraining

- Prior to this discovery, only convolutional deep networks or networks whose depth resulted from recurrence were regarded as feasible to train.
- Today, we now know that greedy layer-wise pretraining is not required to train fully connected deep architectures,
- but unsupervised pretraining approach was the first method to succeed.
- Greedy layer-wise pretraining is called greedy because it is a greedy algorithm
- meaning that it optimizes each piece of the solution independently, one piece at a time, rather than jointly optimizing all pieces

Greedy Layer-Wise Unsupervised Pretraining

- It is called layer-wise because these independent pieces are the layers of the network.
- Greedy layer-wise pretraining proceeds one layer at a time, training the k -th layer while keeping the previous ones fixed.
- the lower layers (which are trained first) are not adapted after the upper layers are introduced.
- It is called unsupervised because each layer is trained with an unsupervised representation learning algorithm.

Greedy Layer-Wise Unsupervised Pretraining

- However it is also called pretraining, because it is supposed to be
- only a first step before a joint training algorithm is applied to fine-tune all the layers together.
- it can be viewed as a regularizer (in some experiments, pretraining decreases test error without decreasing training error)

and a form of parameter initialization.

- Greedy layer-wise unsupervised pretraining can also be used as initialization for other unsupervised learning algorithms,
- such as deep autoencoder and probabilistic models with many layers of latent variables.
- Such models include deep belief networks and deep Boltzmann machines

Greedy Layer-Wise Unsupervised Pretraining

```
 $f \leftarrow \text{Identity function}$   
 $\tilde{X} = X$   
for  $k = 1, \dots, m$  do  
     $f^{(k)} = \mathcal{L}(\tilde{X})$   
     $f \leftarrow f^{(k)} \circ f$   
     $\tilde{X} \leftarrow f^{(k)}(\tilde{X})$   
end for  
if fine-tuning then  
     $f \leftarrow \mathcal{T}(f, X, Y)$   
end if  
Return  $f$ 
```

Unsupervised feature learning algorithm L , which takes a training set of examples and returns an encoder or feature function f .

The raw input data is X , with one row per example and $f^{(1)}(X)$ is the output of the first stage encoder on X .

In the case where fine-tuning is performed, we use a learner T which takes an initial function f , input examples X (and in the supervised fine-tuning case, associated targets Y), and returns a tuned function.

The number of stages is m .

Greedy Layer-Wise Unsupervised Pretraining

- When and Why Does Unsupervised Pretraining Work?
- can yield substantial improvements in test error for classification tasks.
- Unsupervised pretraining combines two different ideas. First, it makes use of the idea that the choice of initial parameters for a deep neural network can have
 - a significant regularizing effect on the model (and, to a lesser extent, that it can improve optimization).
- Second, it makes use of the more general idea that learning about the input distribution can help to learn about the mapping from inputs to outputs.

Greedy Layer-Wise Unsupervised Pretraining

- choice of initial parameters for a deep neural network can have a strong regularizing effect on its performance.
- a learning algorithm can use information learned in unsupervised phase to perform better in supervised learning stage, is better understood.
- The basic idea is that some features that are useful for the unsupervised task may also be useful for the supervised learning task.
- if we wish to add a linear classifier on top of pretrained features, the features must make the underlying classes linearly separable

Greedy Layer-Wise Unsupervised Pretraining

- unsupervised pretraining as a regularizer, we can expect unsupervised pretraining to be most helpful when the number of labeled examples is very small.
- unsupervised pretraining is likely to be most useful when the function to be learned is extremely complicated.
- Unsupervised learning differs from regularizers like weight decay because it does not bias the learner toward discovering a simple function but rather toward discovering feature functions that are useful for the unsupervised learning task.

Greedy Layer-Wise Unsupervised Pretraining

- Unsupervised pretraining has usually been used to improve classifiers
- reducing test set error
- can act to improve optimization rather than being merely a regularizer
- it can improve both train and test reconstruction error for deep autoencoders.
- Neural network training is non-deterministic, and converges to a different function every time it is run.

Greedy Layer-Wise Unsupervised Pretraining

- Training may halt at a point where the gradient becomes small,
- a point where early stopping ends training to prevent overfitting, or at a point where the gradient is large but it is difficult to find a downhill step due to problems such as stochasticity or poor conditioning of the Hessian.
- Neural networks that receive unsupervised pretraining consistently halt in the same region of function space,
- while neural networks without pretraining consistently halt in another region

References

- 1. Ian Goodfellow and Yoshua Bengio and Aaron Courville, Deep Learning (2016) An MIT Press book, <http://www.deeplearningbook.org>.(T2)
- ,MOHAMED ELGENDY, Deep Learning for Vision Systems
- Skansi S., Introduction to Deep Learning - From Logical Calculus to Artificial Intelligence, 1st Edition, Springer International Publishing, 2018.(T2)
- Buduma N., Fundamentals of Deep Learning, 1st Edition, O Reilly Media, 2016.(R1)
- <http://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Teaching/pdf/Lecture9.pdf>